



UNIVERSITÀ
DEGLI STUDI
DI BRESCIA

DIPARTIMENTO DI INGEGNERIA
DELL'INFORMAZIONE

Corso di Laurea
in Ingegneria Informatica Magistrale

Relazione Finale

**Integrazione di funzionalità di
Trigger-Action Programming nel Gemello
Digitale di una Smart Home**

Relatore: Prof.sa Daniela Fogli

Correlatore: Prof.sa Barbara Barricelli

Correlatore: Ing. Davide Guizzardi

Studente:

Giacomo Golino (719210)

Anno Accademico 2024/2025

Indice

Introduzione	II
1 Titolo cap - 1	1
1.1 Introduzione	1
2 Stato dell'arte	2
2.1 Concetti di Base: Automazioni e Routine	3
2.1.1 Requisiti di una buona automazione	4
2.2 Panoramica sulle piattaforme principali	5
2.2.1 Amazon Alexa	5
2.2.2 Google Home	6
2.2.3 Apple Casa (HomeKit)	7
2.3 Metodi e Applicazioni per la Creazione di Automazioni	8
2.3.1 IFTTT	8
2.3.2 Home Assistant	9
2.3.3 Node-RED	11
2.3.4 Confronto fra le tre piattaforme	12
2.3.5 Approcci nella letteratura scientifica	13
2.3.6 Interfacce Visuali	13
2.3.7 Interfacce Conversazionali	15
2.3.8 Interfacce Multi-modali	17
3 Architettura presa in analisi	21
3.1 Digital Twin	21
Conclusioni	23

Introduzione

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Ut purus elit, vestibulum ut, placerat ac, adipiscing vitae, felis. Curabitur dictum gravida mauris. Nam arcu libero, nonummy eget, consectetur id, vulputate a, magna. Donec vehicula augue eu neque. Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Mauris ut leo. Cras viverra metus rhoncus sem. Nulla et lectus vestibulum urna fringilla ultrices. Phasellus eu tellus sit amet tortor gravida placerat. Integer sapien est, iaculis in, pretium quis, viverra ac, nunc. Praesent eget sem vel leo ultrices bibendum. Aenean faucibus. Morbi dolor nulla, malesuada eu, pulvinar at, mollis ac, nulla. Curabitur auctor semper nulla. Donec varius orci eget risus. Duis nibh mi, congue eu, accumsan eleifend, sagittis quis, diam. Duis eget orci sit amet orci dignissim rutrum.

Nam dui ligula, fringilla a, euismod sodales, sollicitudin vel, wisi. Morbi auctor lorem non justo. Nam lacus libero, pretium at, lobortis vitae, ultricies et, tellus. Donec aliquet, tortor sed accumsan bibendum, erat ligula aliquet magna, vitae ornare odio metus a mi. Morbi ac orci et nisl hendrerit mollis. Suspendisse ut massa. Cras nec ante. Pellentesque a nulla. Cum sociis natoque penatibus et magnis dis parturient montes, nascetur ridiculus mus. Aliquam tincidunt urna. Nulla ullamcorper vestibulum turpis. Pellentesque cursus luctus mauris.

1. Titolo cap - 1

1.1 Introduzione

Da introdurre gli obiettivi e il contesto della tesi, ed eventualmente la sua struttura.

2. Stato dell'arte

In questo capitolo verranno analizzati e approfonditi i **metodi** e le **applicazioni** per la creazione di *automazioni* (chiamate anche *routine*) in ecosistemi *Internet of Things* (IoT). In particolare, verranno affrontate con maggiore attenzione le *smart home* come ecosistemi IoT, andando ad analizzare quali sono le principali piattaforme utilizzate per la creazione di *automazioni*.

Si definiscono **automazioni** o *routine* degli insiemi strutturati di istruzioni che, una volta impostati, consentono di eseguire automaticamente una serie di azioni al verificarsi di determinati eventi (o *trigger*). Tali azioni possono comprendere operazioni ripetitive, come l'accensione o lo spegnimento di dispositivi smart, oppure processi più complessi che coinvolgono diversi servizi, con l'obiettivo finale di semplificare e ottimizzare la gestione dell'ecosistema.

Le **azioni** (*action*) costituiscono la parte esecutiva di una routine: al verificarsi dell'evento scatenante (**trigger**), l'azione definita viene avviata per compiere l'operazione desiderata. Può trattarsi di semplici attività oppure di processi articolati che si integrano con servizi diversi.

All'interno di questo capitolo verranno quindi presentati:

- I concetti chiave relativi alle automazioni/routine e la loro importanza nel contesto IoT.
- Le principali piattaforme sul mercato che consentono di creare routine, nello specifico **Alexa (Amazon)**, **Google Home** e **Apple Casa**.
- Le principali piattaforme utilizzate per il *Trigger-Action Programming*.

2.1 Concetti di Base: Automazioni e Routine

Le *automazioni* o *routine* nel contesto dell'IoT consistono in regole di tipo *trigger-action*, dove un evento scatenante (ad esempio, un orario predefinito, un comando vocale o una condizione ambientale rilevata da un sensore) provoca una o più azioni (come l'accensione di una luce, l'avvio di un elettrodomestico, l'invio di una notifica, ecc.). Questa logica di base, semplice da comprendere, si è rivelata estremamente potente e versatile in quanto:

- Riduce la necessità di interventi manuali da parte dell'utente, automatizzando **task** ripetitivi.
- Consente di *personalizzare* lo spazio domestico in base alle preferenze e alle abitudini di ognuno.
- Si presta a una gestione modulare: i vari eventi e azioni possono essere combinati per creare routine più avanzate.

Alcuni studi hanno evidenziato come la programmazione di tipo *trigger-action* (TAP) si adatti alla maggior parte delle esigenze di automazione espresse dagli utenti [1, 2], risultando intuitiva anche per chi non possiede competenze tecniche avanzate. In particolare, l'analisi di oltre 67.000 programmi condivisi su IFTTT e un test di usabilità condotto su 226 partecipanti conferma che, grazie alla semplicità di combinare in modo flessibile molteplici *trigger* e *action*, la curva di apprendimento rimane bassa, favorendo un'ampia adozione [1, 3].

Nell'immagine qui di seguito (fig. 2.1) vengono mostrati alcuni esempi di quelli che potrebbero essere dei **trigger** e delle **action**.

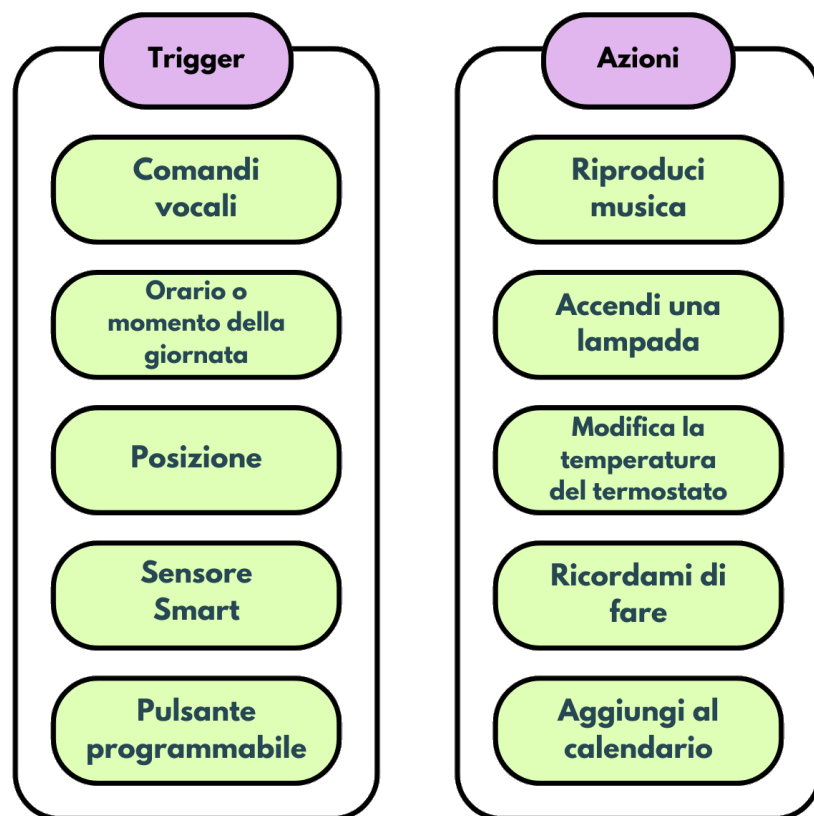


Figura 2.1: Esempio di trigger e azioni

2.1.1 Requisiti di una buona automazione

Per risultare efficace, un'automazione deve essere:

- **Facile da creare e gestire:** l'utente finale (che spesso non possiede competenze di programmazione) deve poter definire e modificare le routine in maniera intuitiva.
- **Affidabile:** deve funzionare in modo consistente nel tempo, senza errori o interruzioni non previste.
- **Adattabile:** dev'essere in grado di gestire modifiche nelle preferenze dell'utente, nelle caratteristiche dei dispositivi o nell'assetto del sistema.
- **Sicura:** in un ecosistema connesso, la *privacy* e la *sicurezza* di reti e dispositivi sono fondamentali.

2.2 Panoramica sulle piattaforme principali

Storicamente, le automazioni si basavano principalmente su *timer* o sensori semplici (come termostati o rilevatori di movimento) per poter funzionare. Al giorno d'oggi, invece, le piattaforme commerciali (e non) offrono soluzioni ben diverse rispetto a quelle di una volta. Nello specifico attualmente vengono utilizzate:

- Interfacce grafiche *drag-and-drop* per comporre in modo visivo le regole.
- Integrazione con *assistenti vocali* (es. Amazon Alexa, Google Assistant e Apple Siri) per impostare routine con frasi naturali.
- Utilizzo di tecniche di *machine learning* e *context awareness* per proporre automazioni “intelligenti” o per adattare le routine al comportamento dell'utente.

Il panorama degli strumenti per la creazione di automazioni è estremamente ampio. In particolare, i sistemi **Amazon Alexa**, **Google Home** e **Apple Casa** sono i principali ecosistemi commerciali per la gestione domestica, ciascuno caratterizzato da specifiche architetture, protocolli di comunicazione e modelli di integrazione con dispositivi di terze parti. Questi principali sistemi sono stati frutto di studi nell'ottica di effettuare un'analisi comparativa per quanto riguarda l'integrazione con dispositivi IoT. [4]

2.2.1 Amazon Alexa

Amazon Alexa è un'assistente vocale lanciato inizialmente su dispositivi *Echo*, poi rapidamente esteso a numerosi dispositivi di terze parti. Le *routine* di Alexa possono essere create tramite:

- L'app Alexa su smartphone.
- Interazione vocale diretta (nello specifico grazie alla sottoscrizione del servizio *Alexa+*, disponibile al momento solo negli USA).
- *Skill* aggiuntive, sviluppate da terze parti.

Inoltre, negli ultimi anni, Amazon ha introdotto:

- **Riconoscimento di suoni specifici:** ad esempio, la piattaforma è in grado di distinguere il suono di vetri rotti, avvisando di conseguenza l'utente. È importante sottolineare che questa funzionalità è disponibile al momento solamente su **Echo smart speakers** e sui dispositivi **Echo smart displays**.
- **Hunches:** questa funzionalità, se attivata attraverso l'applicazione, consente ad Alexa di apprendere determinate routine ricorrenti, così da poter avvisare l'utente qualora tali azioni non vengano eseguite. Ad esempio, Alexa potrebbe imparare che ogni volta che un utente esce di casa, è solito spegnere tutte le luci e chiudere la porta. Nel caso in cui l'utente dovesse dimenticare di svolgere una o entrambe le azioni, il sistema lo notificherebbe, chiedendo se è necessario eseguire queste azioni.
- **Integrazioni con servizi esterni:** questa funzionalità consente l'integrazione con servizi di terze parti come calendari, servizi di musica in streaming o applicazioni di messaggistica.

2.2.2 Google Home

Google Home si basa sull'assistente vocale Google Assistant, sfruttando l'ampia rete di servizi Google. Le automazioni possono derivare dall'utilizzo di:

- Comandi vocali per la creazione e gestione delle *routine*. Gli utenti possono avviare la configurazione di una routine semplicemente pronunciando frasi come "Ok Google, crea una routine per...", dopodiché Google Assistant guiderà attraverso i passaggi per impostare un *trigger* e una o più *action*. Questo metodo semplifica notevolmente l'automazione per chi preferisce un'interazione naturale senza dover passare obbligatoriamente da un'applicazione.
- L'app Google Home, che permette di definire *trigger* e *action* utilizzando dispositivi compatibili.
- L'integrazione con Google Calendar o Google Maps (permettendo, ad esempio, attivare una routine se si è vicini a casa).

L'ecosistema Google si è evoluto introducendo:

- **Eventi geolocalizzati (geofencing):** la routine si attiva o disattiva in base alla posizione dell'utente. Il *geofencing* è una tecnologia basata sulla geolocalizzazione che permette di definire un'area virtuale attorno a una posizione fisica (come casa ad esempio). Quando il dispositivo dell'utente entra o esce da questa zona predefinita, viene attivata una routine automatica. Ad esempio, si può configurare l'accensione delle luci smart quando si arriva a casa o la disattivazione del riscaldamento quando si lascia l'abitazione.
- **Riconoscimento vocale avanzato (Voice Match):** l'assistente riconosce la voce di diverse persone e personalizza alcune automazioni (ad esempio, la musica preferita) in base al profilo.

2.2.3 Apple Casa (HomeKit)

Apple Casa è l'applicazione di Apple per la gestione della domotica, basata sulla piattaforma HomeKit. Rispetto a soluzioni più aperte, punta molto sulla semplicità d'uso e su elevati standard di privacy e sicurezza. Di seguito sono riportate alcune delle principali caratteristiche di questa piattaforma:

- **App Casa su iOS:** qui si impostano automazioni basate sull'orario, sul rilevamento di un sensore, o sull'entrata/uscita da una determinata area geografica.
- **Supporto a scene e stanze:** l'utente può creare "scene" (insiemi di azioni) o associare dispositivi a stanze, semplificando la gestione.
- **Apertura verso standard di connettività:** Apple ha recentemente esteso il supporto a protocolli come Matter, garantendo più integrazione con prodotti non-Apple.

Le ultime versioni di iOS hanno introdotto:

- **Rilevamento della presenza via Apple Watch:** se l'utente indossa un Apple Watch, il sistema può capire se si trova effettivamente in casa (o nei dintorni).

2.3 Metodi e Applicazioni per la Creazione di Automazioni

Oltre alle piattaforme citate, esistono molteplici approcci al *Trigger-Action Programming*, come visto in [2, 3, 4]. Nei seguenti sottocapitoli verrà proposta un'analisi delle caratteristiche principali delle piattaforme più famose che permettono la *Trigger-Action Programming*.

2.3.1 IFTTT

IFTTT (If This Then That) è una delle prime piattaforme di automazione su larga scala, nota per la sua interfaccia intuitiva e la capacità di integrare servizi eterogenei senza richiedere competenze tecniche avanzate [1]. È possibile lavorare con questa piattaforma direttamente dal web, accedendo al sito: ifttt.com.

Gli utenti possono creare semplici regole *IF-THEN* (se accade un evento (*IF*), allora esegui un'azione (*THEN*)) selezionando trigger e azioni predefinite. Ad esempio, si può configurare l'invio automatico di un'email quando viene pubblicato un nuovo post su un blog. Tuttavia, rispetto ad altre soluzioni, IFTTT presenta limitazioni in termini di personalizzazione e controllo, e alcune funzionalità avanzate sono disponibili solo nella versione a pagamento.

In (fig. 2.2) viene mostrato un esempio della creazione di un'automazione che, al verificarsi di una specifica condizione meteo (*trigger*) invia una notifica all'utente (*action*).

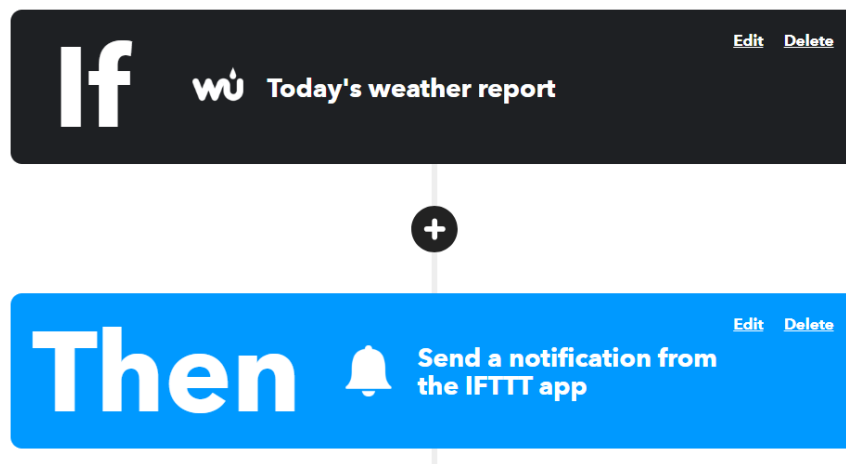


Figura 2.2: Esempio di creazione di una automazione con l'ausilio di IFTTT

2.3.2 Home Assistant

Home Assistant è una piattaforma open-source per la gestione avanzata della domotica, particolarmente apprezzata dagli utenti più esperti per la sua elevata configurabilità e il supporto a un'ampia gamma di dispositivi. È possibile reperire la documentazione di questa piattaforma sul sito: www.home-assistant.io.

Le automazioni possono essere create sia tramite un'interfaccia grafica (Graphic User Interface, GUI) che mediante codice YAML, consentendo logiche più sofisticate rispetto a soluzioni come IFTTT. Nell'esempio riportato in (fig. 2.3), è possibile vedere come è stato possibile programmare una sequenza di eventi condizionali, come l'accensione delle luci in una particolare stanza quando un sensore di un garage rileva l'apertura del suddetto, solo se il sole è già calato, mediante l'interfaccia grafica di Home Assistant.

2. Stato dell'arte

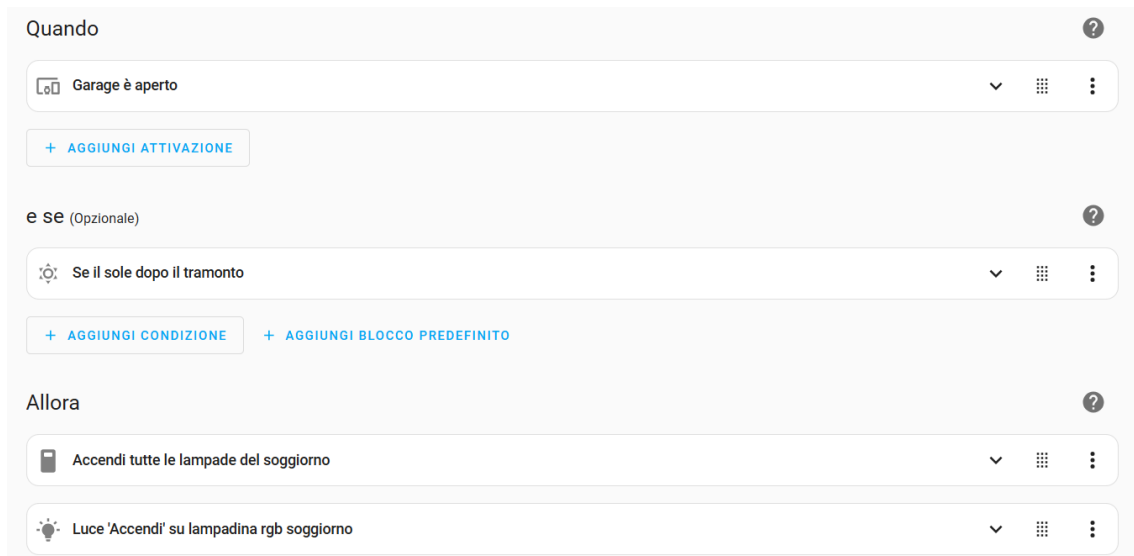


Figura 2.3: Esempio di creazione di un'automazione mediante GUI

La creazione di automazioni mediante codice YAML può risultare però più complessa per degli utenti non esperti, rispetto a quella cloud-based. Nell'esempio di seguito riportato in (fig. 2.4) è illustrato come sia possibile definire un'automazione basata sulla geolocalizzazione degli utenti. In particolare, l'evento scatenante (**trigger**) si verifica quando uno dei dispositivi tracciati cambia stato da **not_home** a **home**, attivando così le azioni corrispondenti dopo un ritardo di un minuto.

```
automation:
  triggers:
    - trigger: state
      entity_id:
        - device_tracker.paulus
        - device_tracker.anne_thereise
      # Optional
      from: "not_home"
      # Optional
      to: "home"
      # If given, will trigger when the condition has been true for X time; you can also use days and
      milliseconds.
      for:
        hours: 0
        minutes: 1
        seconds: 0
```

Figura 2.4: Esempio di codice YAML per la creazione di un'automazione

2.3.3 Node-RED

Node-RED è un ambiente di sviluppo visuale basato su flussi, che consente di creare automazioni collegando elementi modulari chiamati “nodi”. Ogni nodo rappresenta un trigger, un’elaborazione o un’azione, e può essere connesso agli altri in un diagramma di flusso grafico. Questo approccio offre un controllo più avanzato rispetto a IFTTT e una maggiore semplicità rispetto alla configurazione testuale di Home Assistant. Ad esempio, un nodo può ricevere dati da un sensore di temperatura, elaborarli con una soglia personalizzata e inviare una notifica solo se viene superato un valore critico. Tuttavia, essendo una piattaforma più tecnica, richiede una certa familiarità con concetti di programmazione e rete. Qui di seguito, nella (fig. 2.5), viene riportato un flusso utilizzato per la generazione di un’automazione.

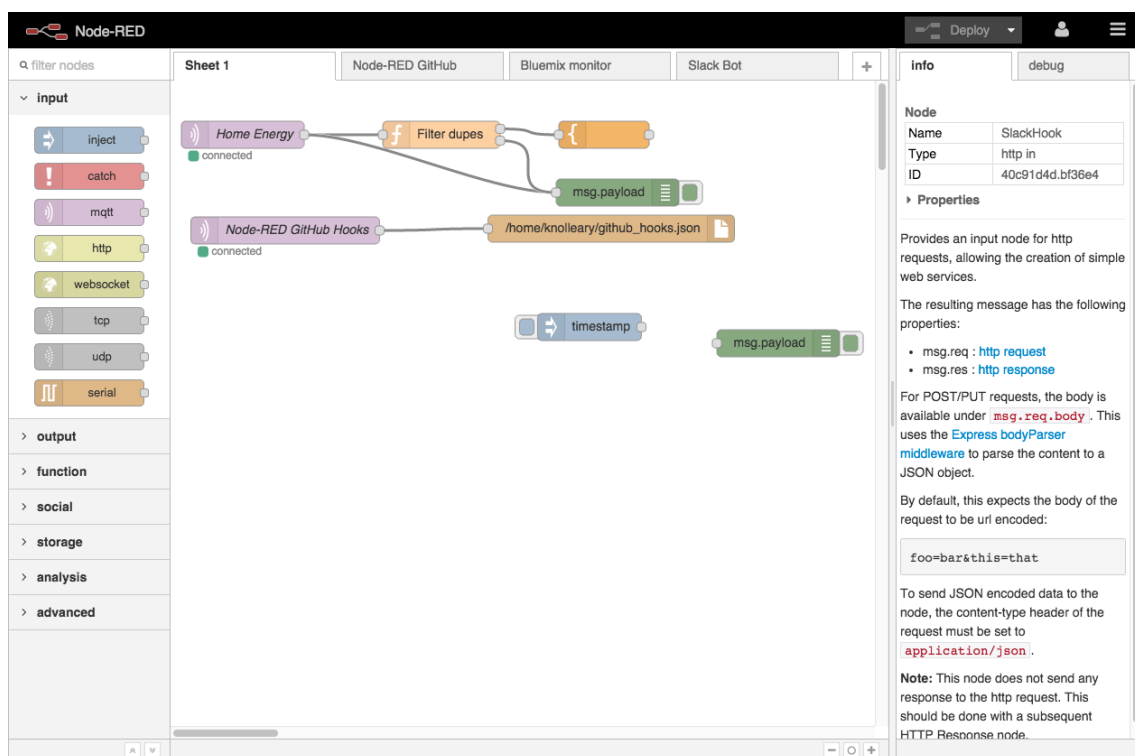


Figura 2.5: Esempio di generazione di un’automazione con Node-RED

2.3.4 Confronto fra le tre piattaforme

Per confrontare IFTTT, Home Assistant e Node-RED sono state considerate metriche analizzate in [5, 6, 7] come:

- a) **Facilità d'uso e curva di apprendimento**
- b) **Potenza espressiva delle regole**
- c) **Modalità di esecuzione (cloud o locale)**
- d) **Ampiezza delle integrazioni disponibili**
- e) **Costo di adozione**

Studi recenti sottolineano come gli strumenti interamente visuali (es. IFTTT) favoriscano la rapidità di prototipazione, ma impongano limiti man mano che cresce la complessità logica [5]. Soluzioni a flusso come Node-RED colmano questo gap offrendo un controllo granulare senza richiedere la completa transizione a un approccio testuale; Home Assistant si colloca a metà strada grazie alla combinazione di un'interfaccia grafica con la possibilità dell'utilizzo di YAML, mantenendo una soglia di ingresso relativamente accessibile ma consentendo automazioni avanzate [6, 7].

Criterio	IFTTT	Home Assistant	Node-RED
Facilità d'uso	Alta	Media	Media-bassa
Potenza logica	Bassa — singola regola	Alta — automazioni / script	Alta — flussi arbitrari
Esecuzione	Cloud	Locale / Cloud	Locale
Integrazioni	~ 700 servizi cloud	~ 2500 integrazioni	~ 3500 nodi
Costo	Freemium	Open-source	Open-source

Tabella 2.1: Confronto fra IFTTT, Home Assistant e Node-RED

In sintesi, *IFTTT* si distingue per la rapidità con cui consente di realizzare automazioni semplici; *Home Assistant* eccelle nei casi domestici complessi grazie all'ampio catalogo di integrazioni e alla combinazione di interfaccia grafica e configurazioni testuali; *Node-RED* offre la massima flessibilità architetturale, risultando ideale per scenari IoT multi-dominio che richiedono logiche di flusso elaborate.

2.3.5 Approcci nella letteratura scientifica

Negli ultimi anni, la ricerca si è concentrata su come rendere accessibile la creazione di routine e flussi di automazione anche a quegli utenti che non possiedono competenze di programmazione (*End-User Development*, EUD). In particolare, sono emersi due approcci complementari per aiutare l'utente nella definizione di regole e workflow: le *interfacce visuali* e le *interfacce conversazionali* [2].

In seguito si è poi deciso di provare a combinare i punti di forza di entrambi gli approcci dando vita alle *interfacce multimodali*, che consentono di sfruttare simultaneamente modalità di input e output sia visuali sia conversazionali, offrendo così un'esperienza più ricca e flessibile per l'utente [4].

2.3.6 Interfacce Visuali

Le interfacce visuali hanno come principale vantaggio la rappresentazione esplicita dei *trigger* e delle *action*, tramite blocchi e connettori, o attraverso grafiche facilmente riconoscibili dall'utente finale. Paradigmi come il *flow-based programming* e il *block-based programming* permettono di trascinare e collegare componenti, rendendo chiari i passaggi di input e output e minimizzando gli errori di configurazione. Inoltre, rappresentazioni iconiche (ad esempio una lampadina o un termostato) possono semplificare l'interazione e offrire un riscontro immediato di ciò che si sta progettando.

Un esempio di creazione di un'automazione mediante due interfacce visuali diverse viene riportato in (fig. 2.6) [2]

2. Stato dell'arte

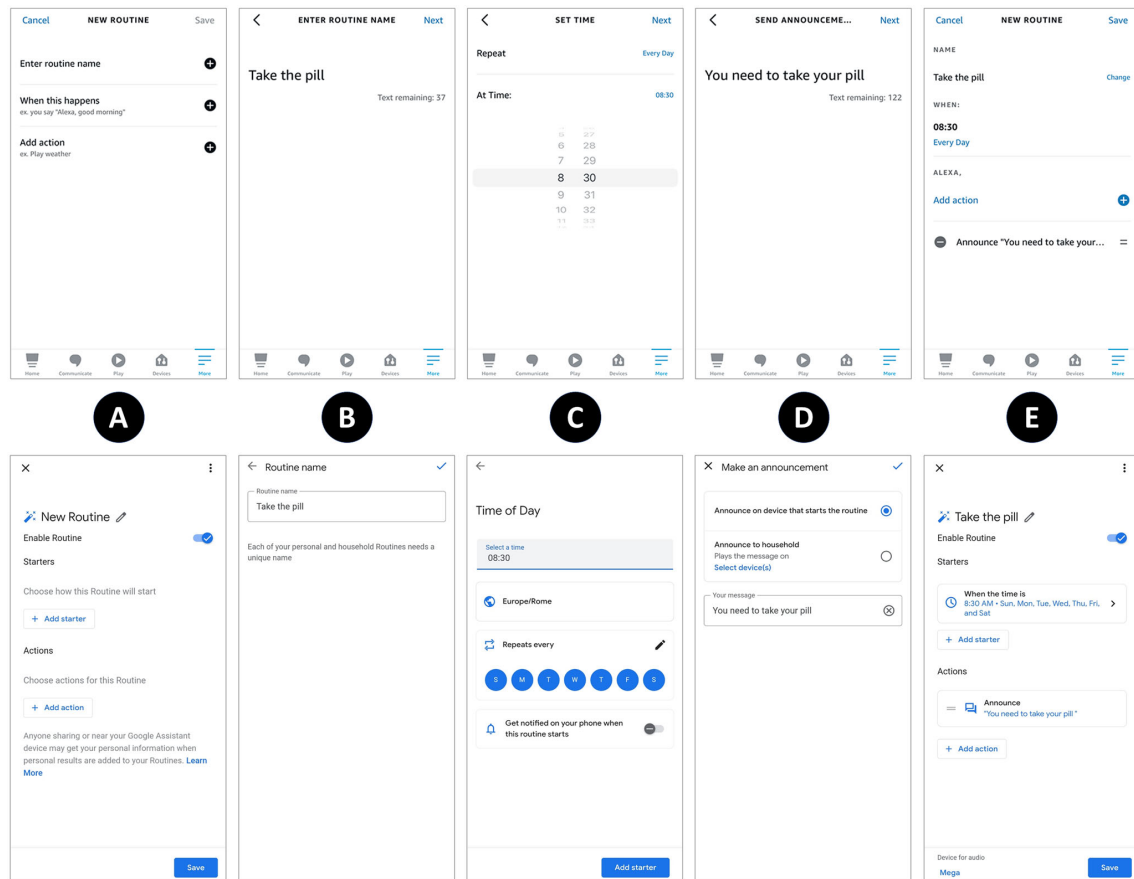


Figura 2.6: Creazione di una routine mediante l'utilizzo dell'applicazione Amazon Alexa (riga superiore) e dell'app Google Home (riga inferiore)

Vantaggi Gli editor grafici basati offrono una *bassa soglia di ingresso* per gli utenti non programmatori: negli studi controllati il tempo di apprendimento medio è inferiore a 15 minuti [8]. Nel progetto AppsGate, gli utenti coinvolti hanno definito la programmazione come «estremamente facile», segnalando un immediato senso di successo nella creazione di regole semplici [9].

Svantaggi. All'aumentare della complessità, i benefici iniziali si attenuano: l'espressione di condizioni composte o di regole multi-dispositivo fa crescere tempi di completamento e tassi d'errore [9].

Ambienti di sviluppo come Node-RED (vedasi 2.3.3) mostrano problemi di *scalabilità visiva*: poche decine di nodi superano lo spazio dello schermo, generando

sovraccarico cognitivo e rendendo difficile comprendere le cause di un'azione [10]. Ulteriori limiti emersi riguardano:

- L'ambiguità tra eventi e stati, che confonde gli utenti [10].
- La mancanza di supporto multi-utente e di simulazioni energetiche, evidenziata da Caivano nel confronto Atooma/IFTTT/Tasker [11].
- L'assenza di meccanismi di risoluzione conflitti: loop, collisioni e ridondanze possono produrre comportamenti inattesi o sprechi energetici [12, 13].

In sintesi, le interfacce visuali rimangono efficaci per regole semplici e utenti alle prime armi, ma necessitano di funzioni di astrazione, filtraggio e analisi per restare usabili in scenari reali e dinamici.

2.3.7 Interfacce Conversazionali

Parallelamente all'approccio visuale, diversi studi hanno approfondito l'efficacia delle cosiddette *interfacce conversazionali* [2], che consentono la definizione di routine mediante interazione in linguaggio naturale con assistenti vocali (ad es. Amazon Alexa, Google Assistant, Apple Siri). In questo caso, la creazione di una regola o routine avviene attraverso un dialogo guidato, dove l'utente può specificare trigger e azioni utilizzando frasi colloquiali [2, 14].

L'utilizzo del linguaggio naturale può apparire più intuitivo rispetto a un'interfaccia a blocchi, specialmente per utenti poco esperti, poiché non richiede di apprendere una sintassi formale o di manipolare elementi grafici. Tuttavia, la sfida maggiore risiede nella corretta interpretazione delle richieste (*Natural Language Understanding*, *NLU*) e nella necessità di meccanismi di conferma o riepilogo, che riducano il rischio di errori o ambiguità.

Esempio di interazione solo vocale:

Utente: “Alexa, voglio creare una nuova routine.”

Assistente: “Va bene, come desideri attivare la routine? Puoi impostare un comando vocale, un giorno e orario specifici, oppure l'alba o il

tramonto.”

Utente: “Imposta il comando vocale.”

Assistente: “Hai scelto di attivare la routine tramite un comando vocale. Dimmi la frase che vuoi usare per avviarla.”

Utente: “Accendi luci salotto.”

Assistente: “Perfetto, hai impostato il comando vocale ‘Accendi luci salotto.’ Ora, desideri aggiungere una o più azioni alla routine?”

Utente: “Sì, aggiungi il comando di accendere le luci del salotto.”

Assistente: “Aggiungo l’azione ‘accensione luci salotto.’ Vuoi aggiungere altro o salvare la routine?”

Utente: “Salva routine.”

Assistente: “La tua routine è stata salvata con il comando vocale ‘Accendi luci salotto.’ Puoi modificarla in qualsiasi momento.”

In questo esempio [2], si evidenzia come l’assistente debba fornire indicazioni chiare per ciascun passaggio, ripetere o confermare le scelte dell’utente e limitare la complessità di ogni richiesta per evitare sovraccarico cognitivo. Questo approccio è coerente con i principi di progettazione conversazionale e con la cosiddetta *Miller’s Law*, che evidenzia come la capacità di memorizzazione e comprensione degli utenti sia limitata a un numero ristretto di elementi per volta.

Vantaggi

L’interazione in linguaggio naturale consente un accesso “senza mani” e a bassa soglia cognitiva: negli esperimenti con smart speaker *speech-only*, il tasso di completamento delle routine semplici ha raggiunto il 94 % con un punteggio SUS medio di 78/100 [10]. Jarvis (assistente conversazionale progettato per la gestione di sistemi IoT complessi tramite linguaggio naturale [10]) mostra che la formulazione vocale riduce il tempo medio di espressione di una regola del 30 % rispetto alla costruzione grafica in Node-RED (2.3.3) [10].

L'impiego di chatbot basati su modelli linguistici di grandi dimensioni (es. RuleBot++) migliora la comprensione da parte del sistema delle richieste effettuate dall'utente [14].

Inoltre gli assistenti vocali risultano inclusivi per utenti con ridotta capacità visiva o con mani occupate, ampliando l'accessibilità del controllo domestico [15].

Svantaggi

- **Ambiguità semantiche:** la stessa frase può essere interpretata in modi diversi; nelle prove di Jarvis oltre il 20 % dei comandi liberi ha richiesto chiarimenti [10].
- **Recupero da errori:** senza un supporto visivo, gli utenti faticano a capire che cosa non sia stato compreso e come riformulare; ParlAmI riporta interazioni interrotte nel 15 % dei casi [16].
- **Scalabilità logica:** con regole che includono più di due trigger e tre azioni aumentano i turni di dialogo; RuleBot++ registra un +42 % di turni di chiarimento rispetto a regole elementari [14].
- **Privacy e fiducia:** nei diari d'uso longitudinali di Sciuto et al. il 38 % degli utenti ha limitato i comandi sensibili per timore di ascolto continuo [15].

In sintesi, le interfacce conversazionali semplificano l'accesso e rendono naturale la definizione di automazioni semplici; per scenari complessi occorrono strategie di disambiguazione, conferma e sintesi per mantenere affidabilità ed efficienza.

2.3.8 Interfacce Multi-modali

Per far fronte alle problematiche che possono nascere dall'utilizzo di un'interfaccia puramente conversazionale o visuale è possibile andare ad utilizzare quelle che vengono definite *interfacce multi-modali*. Con questo termine si identificano quei dispositivi che permettono di interagire con il sistema sfruttando simultaneamente più canali comunicativi, come ad esempio voce e visualizzazione di un'interfaccia grafica. [4, 16]

Un'evoluzione di questo filone è **RuleBot ++** [14], un chatbot basato su ChatGPT (GPT-4) integrato in un'interfaccia multi-modale: l'utente può impartire i comandi a voce o via chat, visualizzare la regola generata sul display (o in app) e, se necessario, correggerla toccando i singoli parametri. Nei test di usabilità (16 partecipanti) RuleBot ++ ha fatto registrare un punteggio SUS medio di 78/100 e un *time-on-task* inferiore del 35 % rispetto alla GUI di Home Assistant.

In (fig. 2.7) è possibile vedere un esempio di interfaccia multi-modale utilizzata per la definizione di una nuova routine tramite l'ausilio di un dispositivo *Amazon Alexa*. [4]



Figura 2.7: Esempio di un sistema multi-modale utilizzato per la generazione di routine

Una recente analisi sperimentale condotta in [4] ha evidenziato i vantaggi dell'utilizzo di soluzioni multi-modali per la creazione di routine all'interno di ecosistemi IoT, in quanto:

- Le interfacce multi-modali risultano più apprezzate dagli utenti rispetto alle interazioni *voice-only*, specialmente per la possibilità di visualizzare in tempo reale le scelte già effettuate e selezionare i comandi su schermo.
- Le interfacce conversazionali puramente vocali mantengono un elevato grado di accessibilità, ma richiedono una maggiore attenzione e memoria da parte dell'utente per seguire l'intero dialogo ed evitare di perdersi tra le opzioni disponibili.
- Aspetti come la dipendenza dal brand (ad esempio, la familiarità con Alexa o con Google Assistant) e il livello di esperienza con i dispositivi smart possono influire significativamente sulla percezione di usabilità e soddisfazione, sugge-

rendo che la personalizzazione dell'interfaccia e la coerenza del dialogo siano fattori chiave.

Vantaggi

- **Integrazione voce e display.** Quando input vocali e riscontri grafici/tattili convivono, i limiti del canale singolo si riducono sensibilmente: l'utente può parlare per avviare la routine e, nello stesso tempo, verificare a colpo d'occhio ciò che il sistema ha compreso.
- **Evidenze sperimentali** [4]. Sui dispositivi Echo Show la modalità multi-modale ha portato il tasso di completamento dal 88 % (solo voce) al 97 % e innalzato il punteggio SUS da 74/100 a 83/100; ciò conferma che la doppia modalità migliora sia efficacia sia soddisfazione.
- **Maggiore controllo.** I partecipanti allo studio hanno apprezzato di poter usare il display per rivedere o correggere parametri complessi, lasciando alla voce i comandi rapidi («Hey Alexa, salva»).[4]
- **Accessibilità amplificata.** La ridondanza dei canali favorisce pubblici diversi: persone con disabilità visive possono affidarsi alla sintesi vocale, mentre chi opera in ambienti rumorosi o con le mani occupate ricorre all'interfaccia touch.
- **Supporto “intelligente” alla correzione.** Integrando un LLM, come nel caso di *RuleBot ++*, l'interfaccia è in grado di proporre suggerimenti o versioni alternative della routine, riducendo errori sintattici e velocizzando la rifinitura delle regole.[14]

Svantaggi

- **Aumento della complessità dell'UI:** alcuni utenti hanno segnalato confusione sul “dove guardare” quando la creazione di una routine visualizzava contemporaneamente conferme testuali e vocali [2].
- **Costi e frammentazione:** display intelligenti e robot aumentano i costi

di adozione e introducono differenze tra ecosistemi (Amazon vs Google) che richiedono percorsi di progettazione paralleli [2].

- **Gestione dei conflitti multimodali:** ParlAmI riporta episodi di competizione tra input vocali e tocchi simultanei, con necessità di strategie di arbitraggio per evitare comandi duplicati [16].
- **Maggiore carico cognitivo in scenari complessi:** quando la regola supera tre azioni, il passaggio continuo voce touch aumenta il tempo medio di completamento del 22 % rispetto alla sola interazione touch [2].

3. Architettura presa in analisi

In questo capitolo verrà analizzata l'architettura del sistema utilizzato durante tutto il caso di studio di questo elaborato. Nello specifico, in figura (3.1) possiamo vedere una prima visualizzazione del sistema preso in analisi.

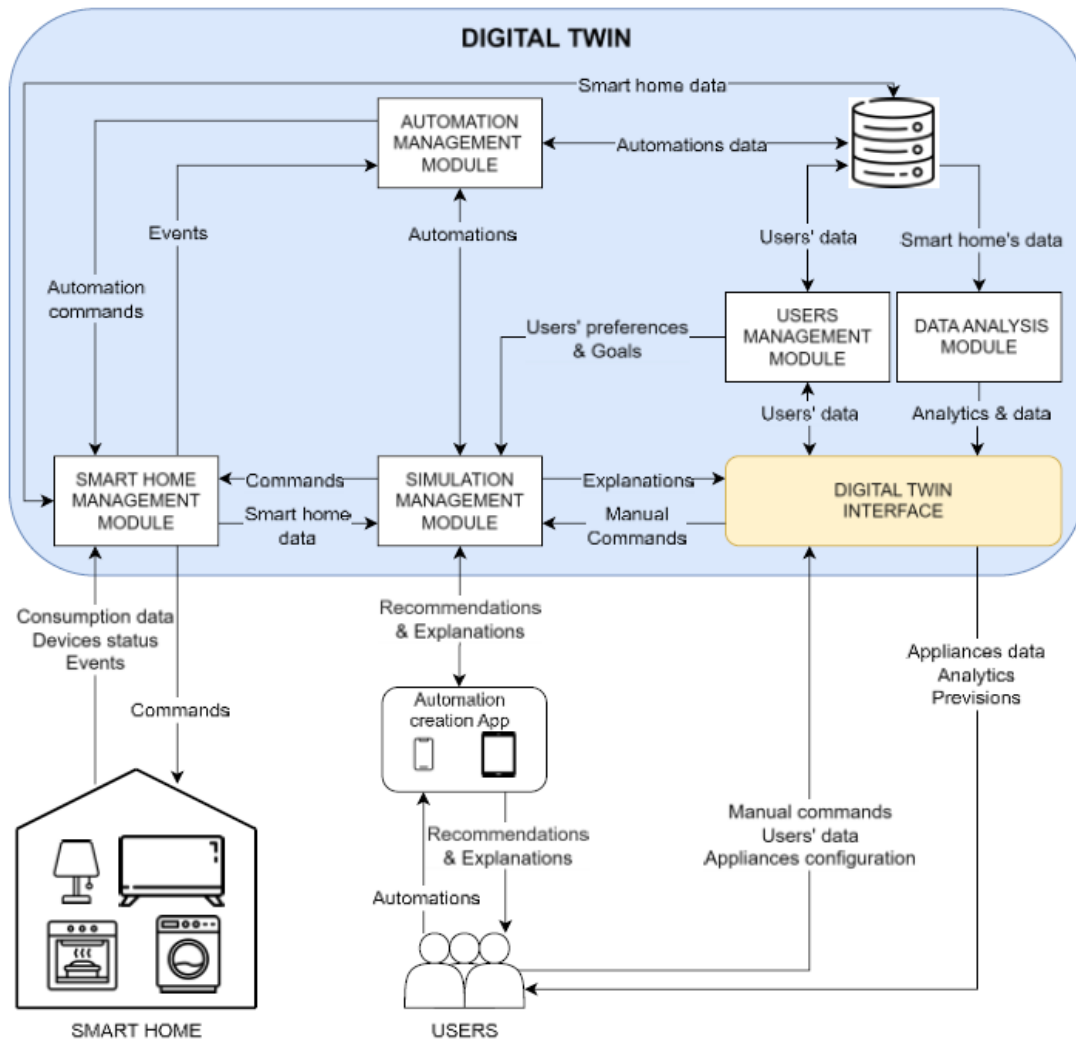


Figura 3.1: Architettura di riferimento

3.1 Digital Twin

Il *Digital Twin* (DT) della smart-home è stato concepito come una **replica digitale persistente e sincronizzata** che riproduce in tempo quasi-reale lo stato di

dispositivi, ambienti e routine automatizzate (*Trigger-Action*) dell'abitazione. Gli obiettivi principali sono:

- ridurre il rischio di configurazioni errate validando le regole TAP in un ambiente sicuro;
- fornire analisi predittive su consumo energetico e comfort;
- supportare un'interfaccia utente per la visualizzazione degli effetti delle automazioni.

Conclusioni

Fusce mauris. Vestibulum luctus nibh at lectus. Sed bibendum, nulla a faucibus semper, leo velit ultricies tellus, ac venenatis arcu wisi vel nisl. Vestibulum diam. Aliquam pellentesque, augue quis sagittis posuere, turpis lacus congue quam, in hendrerit risus eros eget felis. Maecenas eget erat in sapien mattis porttitor. Vestibulum porttitor. Nulla facilisi. Sed a turpis eu lacus commodo facilisis. Morbi fringilla, wisi in dignissim interdum, justo lectus sagittis dui, et vehicula libero dui cursus dui. Mauris tempor ligula sed lacus. Duis cursus enim ut augue. Cras ac magna. Cras nulla. Nulla egestas. Curabitur a leo. Quisque egestas wisi eget nunc. Nam feugiat lacus vel est. Curabitur consectetur.

Bibliografia

- [1] Blase Elyse Ur, Elyse McManus Ho, Melwyn Pak Yong Ho e Michael L. Littman. «Practical Trigger-Action Programming in the Smart Home». In: *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems (CHI '14)*. New York, NY, USA: Association for Computing Machinery, 2014, pp. 803–812. ISBN: 9781450324731. DOI: 10.1145/2556288.2557420. URL: <https://doi.org/10.1145/2556288.2557420>.
- [2] Barbara Rita Barricelli, Alessandro Bondioli, Daniela Fogli, Letizia Iemmolo e Angela Locoro. «Creating Routines for IoT Ecosystems through Conversation with Smart Speakers». In: *International Journal of Human-Computer Interaction* 40.20 (2024), pp. 6109–6127. DOI: 10.1080/10447318.2023.2247845. URL: <https://doi.org/10.1080/10447318.2023.2247845>.
- [3] Fulvio Corno, Luigi De Russis e Alberto Monge Roffarello. «From Users' Intentions to IF-THEN Rules in the Internet of Things». In: *ACM Transactions on Information Systems* 39.4 (2021), 53:1–53:33. DOI: 10.1145/3447264. URL: <https://doi.org/10.1145/3447264>.
- [4] Barbara Rita Barricelli, Daniela Fogli, Letizia Iemmolo e Angela Locoro. «A Multi-Modal Approach to Creating Routines for Smart Speakers». In: *Proceedings of the 2022 International Conference on Advanced Visual Interfaces (AVI '22)*. Association for Computing Machinery, 2022, pp. 1–5. DOI: 10.1145/3531073.3531168. URL: <https://doi.org/10.1145/3531073.3531168>.
- [5] McKenna McCall, Eric Zeng, Faysal Hossain Shezan, Mitchell Yang, Lujo Bauer, Abhishek Bichhawat, Camille Cobb, Limin Jia e Yuan Tian. «Towards Usable Security Analysis Tools for Trigger-Action Programming». In: *Proceedings of the Nineteenth Symposium on Usable Privacy and Security (SOUPS 2023)*. A cura di Patrick G. Kelley e Apu Kapadia. Anaheim, CA: USENIX Association, ago. 2023, pp. 301–320. ISBN: 978-1-939133-36-6. URL: <https://www.usenix.org/system/files/soups2023-mccall.pdf>.

3. BIBLIOGRAFIA

- [6] Likewin Thomas, M. K. Manoj Kumar, S. D. Shiva Darshan e B. S. Prashanth. «Towards Comprehensive Home Automation: Leveraging the IoT, Node-RED, and Wireless Sensor Networks for Enhanced Control and Connectivity». In: *Engineering Proceedings*. Vol. 59. 2023, p. 173. DOI: 10.3390/engproc2023059173.
- [7] Anders Peter Aavild, Aleksander Rosenkrantz de Lasson, Christian Moesgaard, Erik Christensen, Sergio Moreschini, David Hastbacka, Davide Taibi e Michele Albano. «Distributed Home Automation with Home Assistant». In: *Proceedings of the 14th International Conference on the Internet of Things (IoT '24)*. 2024, pp. 206–212. DOI: 10.1145/3703790.3703828.
- [8] Giuseppe Desolda, Carmelo Ardito e Maristella Matera. «Empowering End Users to Customize Their Smart Environments: Model, Composition Paradigms, and Domain-Specific Tools». In: *ACM Transactions on Computer-Human Interaction* 24.2 (2017), 12:1–12:52. DOI: 10.1145/3057859. URL: <https://doi.org/10.1145/3057859>.
- [9] Joëlle Coutaz e James L. Crowley. «A First-Person Experience with End-User Development for Smart Homes». In: *IEEE Pervasive Computing* 15.2 (2016), pp. 26–36. DOI: 10.1109/MPRV.2016.34. URL: <https://doi.org/10.1109/MPRV.2016.34>.
- [10] André Sousa Lago, João Pedro Dias e Hugo Sereno Ferreira. «Managing non-trivial internet-of-things systems with conversational assistants: A prototype and a feasibility experiment». In: *Journal of Computational Science* 51 (2021), p. 101324. DOI: 10.1016/j.jocs.2021.101324. URL: <https://doi.org/10.1016/j.jocs.2021.101324>.
- [11] Danilo Caivano, Daniela Fogli, Rosa Lanzilotti, Antonio Piccinno e Fabio Casano. «Supporting end users to control their smart home: Design implications from a literature review and an empirical investigation». In: *The Journal of Systems & Software* 144 (2018), pp. 295–313. DOI: 10.1016/j.jss.2018.06.035. URL: <https://doi.org/10.1016/j.jss.2018.06.035>.

- [12] Fulvio Corno, Luigi De Russis e Alberto Monge Roffarello. «Loops, Inconsistencies and Redundancies in Trigger-Action Rules for Smart Environments». In: *Proceedings of the 5th International Conference on Internet of Things Design and Implementation (IoTDI '20)*. New York, NY, USA: Association for Computing Machinery, 2020, pp. 259–270. DOI: 10.1145/3386910.3397254. URL: <https://doi.org/10.1145/3386910.3397254>.
- [13] Xiang Chen, Yuncong Li, Earlene Fernandes e Atul Prakash. «Understanding Rule Preventions, Collisions, and Unexpected Chains in Trigger-Action Programming». In: *Proceedings of the 42nd IEEE Symposium on Security and Privacy (S&P '21)*. IEEE, 2021, pp. 1154–1171. DOI: 10.1109/SP40001.2021.00091. URL: <https://doi.org/10.1109/SP40001.2021.00091>.
- [14] Simone Gallo, Fabio Paternò e Alessio Malizia. «A Conversational Agent for Creating Automations Exploiting Large Language Models». In: *Personal and Ubiquitous Computing* 28.5 (2024), pp. 931–946. DOI: 10.1007/s00779-024-01825-5. URL: <https://doi.org/10.1007/s00779-024-01825-5>.
- [15] Andrea Sciuto, Arnita Saini, Jodi Forlizzi e Jason I. Hong. «"Hey Alexa, What's Up?": A Mixed-Methods Study of In-Home Conversational Agent Usage». In: *Proceedings of the 2018 Designing Interactive Systems Conference (DIS '18)*. ACM, 2018, pp. 857–868. DOI: 10.1145/3196709.3196772.
- [16] Evropi Stefanidi, Michalis Foukarakis, Dimitrios Arampatzis, Maria Korozi, Asterios Leonidis e Margherita Antona. «ParlAmI: A Multimodal Approach for Programming Intelligent Environments». In: *Technologies* 7.1 (2019), p. 11. DOI: 10.3390/technologies7010011. URL: <https://doi.org/10.3390/technologies7010011>.